

Generating Software Architecture Description from Source Code using Reverse Engineering and Large Language Model

1st Ahmad Hatahet ^{*}, 2nd Christoph Knieke [†], 3rd Andreas Rausch [‡],

Institute for Software and Systems Engineering

Technical University of Clausthal

Clausthal-Zellerfeld, Germany

^{*}ahmad.hatahet@tu-clausthal.de , [†]christoph.knieke@tu-clausthal.de , [‡]andreas.rausch@tu-clausthal.de 

Abstract—Software Architecture Descriptions (SADs) are indispensable for navigating the complexity of modern software systems. They enable high-level reasoning, guide design decisions, and facilitate communication among diverse stakeholders. Yet in practice, SADs are frequently missing, outdated, or poorly aligned with the system’s actual implementation. As a result, developers are forced to derive architectural insights directly from source code—a time-intensive task that increases mental effort, slows onboarding, and contributes to the gradual degradation of architectural clarity over a system’s lifetime. To address these challenges, we propose an automated generation of SADs from source code by combining reverse engineering (RE) techniques with Large Language Model (LLM). Our approach recovers both static and behavioral views by extracting a comprehensive component diagram, filters architecturally significant elements—core components—via prompt-guided LLM analysis, and generates state machine diagrams to model component behavior based on code logic. This dual-view representation offers a scalable and maintainable alternative to manual architectural documentation. This approach, demonstrated with C++ examples, showcases the potent capability of LLMs to: 1) find abstract component diagram, lowering the reliance on human expert involvement, and 2) represent complex software behaviors, especially when enriched with domain-specific knowledge through few-shot prompting. These findings suggest a viable path toward reducing manual effort while enhancing system understanding and long-term maintainability.

Index Terms—software architecture, architecture description, architecture recovery, reverse engineering, large language model.

I. INTRODUCTION

Software Architecture (SA) plays a central role in managing complexity during software development by defining the system’s core structure, key components, and their interactions. As a foundational artifact, it guides long-term evolution, supports informed technical decisions, and provides alignment between system requirements and implementation. To fulfill its role effectively, architecture must be explicitly defined and systematically documented.

Software Architecture Description (SAD) provides this systematic formalization [1], capturing architectural knowledge in the form of structured views and artifacts. These views enable communication across stakeholders, from managers to

developers, each requiring a different level of abstraction. In this context, two views are especially critical: the static view, typically presented as component diagrams, which describes structural elements and their relationships; and the behavioral view, often modeled through state machine diagrams, which captures component dynamics and operational lifecycles.

In practice, however, SADs are often missing, incomplete, or outdated [2]–[4]. Teams frequently rely on source code as the sole reference point for understanding system structure and behavior. Yet, source code alone lacks architectural abstraction and is not suited for stakeholder comprehension. This gap leads to increased onboarding effort, reduced design clarity, and a gradual erosion of system-level understanding, especially in real-world and complex projects [4], [5].

To mitigate this gap, reverse engineering (RE) [6] has been widely explored as a means to recover architectural information from source code. These techniques, while useful for extracting structural representations, tend to overproduce low-level details or miss semantically significant connections. Moreover, most solutions are limited to static views, leaving behavioral aspects of the architecture undocumented and non-analyzed [7].

Recent progress in Large Language Models (LLMs) opens a promising path forward. With strong capabilities in code comprehension and natural language generation [8], LLMs can support a range of development tasks, such as: summarizing logic, interpreting behavior, and generating visual diagrams using diagram language like PlantUML [9], [10]. Their ability to generalize across languages and contexts makes them particularly suited to behavior modeling. However, their effectiveness depends on precise instructions and relevant context, such as specific target code scripts to avoid irregular behavioral interpretations.

In this work, we propose a hybrid approach that combines RE and LLM-based reasoning to automate the generation of software architecture descriptions from source code. The process begins with RE to extract a comprehensive class diagram that reflects the system’s structural details. However, this low-level representation must be abstracted into a higher-level component diagram—where each component encapsulates a

cohesive group of classes and exposes well-defined interfaces to other components. While this abstraction step is crucial for architectural relevance, yet it is project-specific and context-dependent. As such, it is delegated to the LLM, which is dynamic and able to interpret code semantics and reorganize the structure into meaningful architectural components [11], [12], thus, it is a well-suited for this role. Once the core components are established, the LLM analyzes their internal logic and generates corresponding state machine diagrams to capture behavioral patterns. The process is designed to maintain semantic accuracy while managing input size constraints.

This paper explores: **RQ1)** to what extent SAD can be automatically recovered from source code alone, structurally and behaviorally, without relying on prior documentation or annotations?. **RQ2)** can an LLM, with scoped input, generate accurate behavioral views (state machine diagrams) that reflect the internal logic of architectural components? Through this investigation, we aim to demonstrate a practical solution to SAD that supports better comprehension, and system transparency.

The rest of the paper is structured as follows. Sec. II reviews the related work, Sec. III describes our approach and methodology in detail, Sec. IV presents our results and discusses them. Finally, conclusions and future research are summarized in Sec. V.

Note: All diagram images, python notebooks, and prompts are available in our GitHub repository [13].

II. RELATED WORK

Learning a software architecture from code involves the task of identifying and recovering the underlying architecture of a software system from its existing code base. The main objective of this process is to create a high-level architectural model, including its components, relationships, and the architectural patterns used in the implementation.

Reverse engineering architectural models from code is a crucial process [6] because it provides a better understanding of a system's structure and the patterns applied. It serves as a valuable source for future decision-making related to maintaining and further developing the system. Manually maintained documentation and models of a system tend to diverge from the actual source code of the system over time, reducing the reliability of such artifacts [4]. In some cases, these documents may not even exist at all.

Static approaches focus on analyzing the source code of a software system without executing it [14]. These techniques use various code analysis tools to extract information from the source code, such as parsing source code to extract syntax and control flow information or using metrics to quantify certain properties of the code [15].

Various tools have attempted to automate architectural recovery, though with notable trade-offs. Tools like Doxygen [16] and SmartDraw [17] generate component diagrams directly from source code. However, these diagrams often omit critical architectural details—such as class attributes, method implementations, and inter-class associations—rendering them

inadequate for comprehensive static views. SmartDraw, in particular, captures associations inconsistently, limiting its reliability.

CLANG-UML [18] improves automation by generating diagrams in PlantUML format [19], yet its output is often too granular. For real-world systems, the resulting diagrams are overwhelmed with low-level implementation details, complicating architectural reasoning. Similarly, Enterprise Architect (EA) [20] supports multi-language RE and rich modeling capabilities, but like its counterparts, struggles to deliver the necessary level of abstraction for architectural-level insight.

To address these gaps, several research efforts propose more tailored or hybrid solutions. Schiewe et al. [14] introduce a language-agnostic methodology for component identification using an intermediate representation—LAAS—guided by heuristic rules. This approach abstracts away syntactic language features, aiming to detect higher-level architectural components. Walker et al. [21], targeting microservice systems, combine static analysis with runtime tracing to reconstruct distributed architectures. This hybrid approach effectively captures both structural and behavioral aspects of the system.

Some efforts are context-specific. Amalfitano et al. [22] propose a domain-tailored solution for embedded systems using C. Their custom tool automates the generation of software architecture documentation, producing package, component, component and connector (C&C), and state machine diagrams. This approach is particularly noteworthy because its success stems from effectively leveraging specific code characteristics inherent to their embedded system use case. For instance, the tool defines the package diagram based on the system's folder structure, and it extracts system states and their triggers from C code references to switch-based task tables. This illustrates how domain-specific knowledge of code organization and patterns can be instrumental in successful architecture recovery.

Other recent efforts pursue automation through graph-based techniques. Zhang et al. [23] merge dependency graphs with directory structures, assigning edge weights and applying clustering to extract architectural components. Puchala et al. [24] follow a similar clustering-based methodology, while Sajji et al. [25] adopt Graph Neural Networks (GNNs) to learn abstraction from structural code representations.

Overall, while numerous tools and methods exist, current techniques often trade off between abstraction and completeness. Effective architecture recovery remains challenging, particularly in ensuring scalability, accuracy, and clarity across diverse codebases and application domains.

III. APPROACH AND METHODOLOGY

This work presents a semi-automated approach for generating software architecture descriptions from source code, focusing on both the static and behavioral views. The process begins by analyzing the source code via RE, producing an initial detailed class diagram. This diagram is translated into PlantUML format using a direct mapping method that captures the class with its attributes and operations, and general associations between the classes.

The LLM receives the full plain PlantUML then it identifies and retains only the architecturally significant classes essential for understanding the system using prompt engineering, we refer to them as core components. The output of the LLM is a list of the core components, which help refine and abstract the class diagram into a component diagram.

For the behavioral view, we generate a state machine diagram for each core component. This is done by providing its corresponding source code to the LLM along with solution plan and few-shot example to infer possible states and transitions based on internal logic and method behaviors.

The result is a dual-view SAD: 1) an abstract component diagram capturing the system's structure and 2) a set of state machine diagrams illustrating key component behaviors. This methodology balances automation with guided abstraction to support system understanding and architectural clarity.

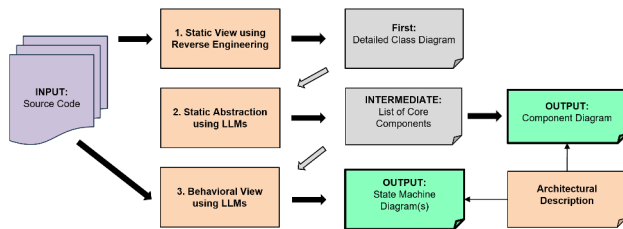


Fig. 1. Proposed approach: the source code is used as an input then after RE, a detailed class diagram generated, after using LLM to abstract it to a component diagram, the LLM also generate multiple state machine diagrams for each component. Component and stat machine diagrams are the output forming the final SAD.

A. Static View using Reverse Engineering

The process begins with extracting a detailed structural representation of the system using Enterprise Architect (EA) [20]. The source code is imported into EA, which statically analyzes the system and generates a comprehensive class diagram. This diagram reads all the classes and their interconnections, forming an exhaustive map of the system's structure, and often includes low-level elements that may obscure the overall architecture. As such, it serves as an initial artifact from which a more meaningful abstraction will be derived in the subsequent step.

B. Static Abstraction using LLM

To address this, EA [20] exports the component diagram to an XML file. Afterward, a custom Python function parses this XML file into PlantUML format, enabling easier manipulation. An example PlantUML snippet is shown below:

PlantUML Example of only two Classes

```
class CoffeeMachine {
    -waterLevel
    +startBrew()
}
class Boiler {
    -temperature
    +heatWater()
}
CoffeeMachine -- Boiler
```

The full PlantUML code is then passed to GPT-4o version 2024-11-20 [26], which identifies architecturally significant classes—referred to as core components—using a system prompt. A second script filters the detailed component diagram in the XML file, retaining only these core components and their relationships to form an abstract structural view.

C. Behavioral View using LLM

For behavioral modeling, the source code of each core component is submitted to GPT-4o version 2024-11-20 [26] alongside few-shot prompting [27]. The examples teach the LLM the link between code and state diagrams. This guides the model in generating state machine diagrams that reflect internal logic and transitions.

Together, these steps produce a refined, dual-view architecture description: a simplified component diagram for structure and targeted state diagrams for behavior. The integration balances automation with selective abstraction, enhancing clarity and architectural relevance. Finally, the evaluation of the generated diagrams is compared with ground truth diagrams to validate the effectiveness of the approach and the quality of the diagrams.

IV. RESULTS AND DISCUSSION

This section presents and analyzes the outcomes of our approach for generating software architecture descriptions from source code, focusing on both the static and behavioral views. The static view is captured in the form of component diagrams, while the behavioral view is expressed through state machine diagrams. We evaluate the quality and fidelity of the generated architectural artifacts against ground truth diagrams using clearly defined criteria and through comparative inspection.

A. Static View: Component Diagram Extraction

To ensure a valid evaluation, our starting point involved selecting systems with available ground truth representations of both class and state machine diagrams. Given the industrial relevance and our collaboration with IAV¹, the source code had to be in C++. Among available tools, IBM Rhapsody² version 8.4 provided a curated set of C++ examples that included architectural documentation. From this set, we selected two illustrative systems: the *Coffee Machine* and the *Dishwasher*.

¹<https://www.iav.com/>

²<https://www.ibm.com/support/pages/ibm-rational-rhapsody-84>

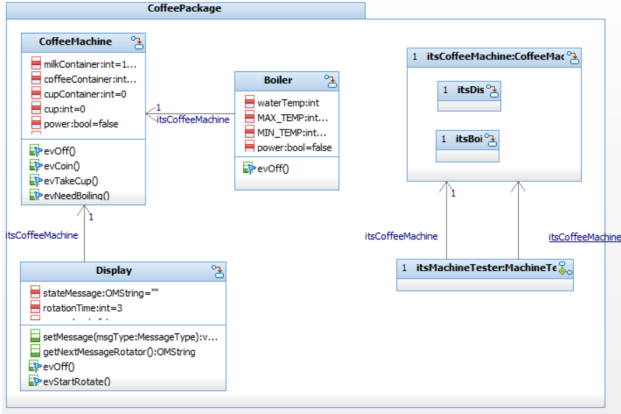


Fig. 2. Ground truth component diagram from Rhapsody showing the core components of the Coffee Machine

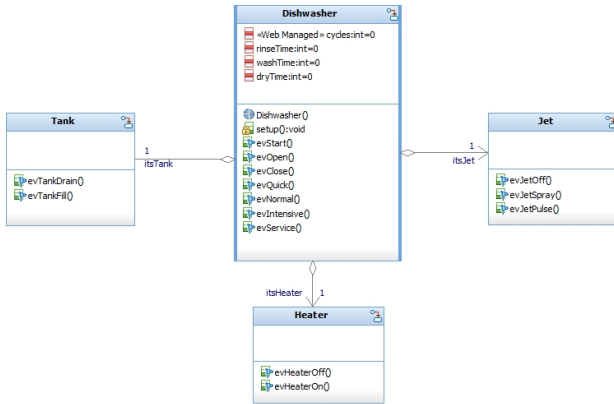


Fig. 3. Ground truth component diagram from Rhapsody showing the core components of the Dishwasher

The reference component diagrams of these systems shown in Fig. 2 and Fig. 3 respectively. We reverse-engineered the corresponding C++ code using EA [20], which resulted in highly detailed and extensive component diagrams as depicted in Fig. 4. These diagrams, while accurate, included numerous auxiliary classes that obscure the essential structural components. Therefore, further abstraction was necessary.

B. Core Component Identification and Abstraction

As described, to facilitate automated reasoning with LLMs, we implemented a naïve but deterministic translation mechanism to convert the EA [20] component diagrams into PlantUML syntax [19]. The translation included all class names, their attributes, operations, and direct associations, preserving the structure observed in the EA [20] generated diagrams.

This PlantUML specification was then passed to GPT-4o version 2024-11-20 [26] with a prompt designed to extract the core architectural components.

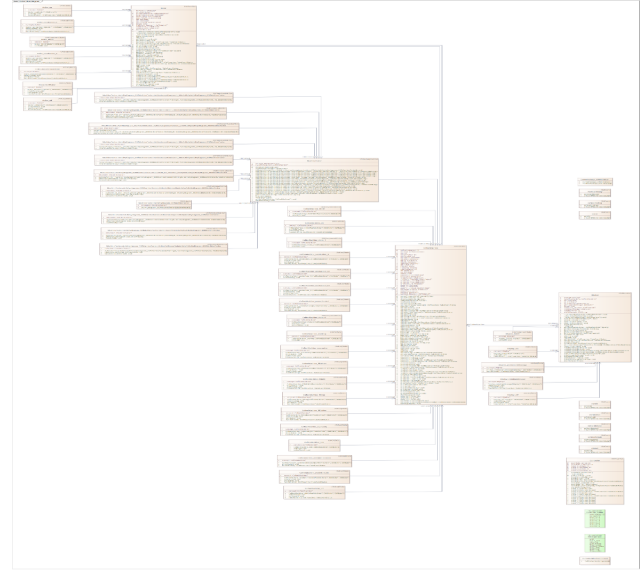


Fig. 4. Overview of the complexity of all Coffee Machine's classes after RE the source code via EA (full resolution available in our repository: [diagrams/CoffeeMachine/class_diagram/class-diagram.png](#))

Find Core Components Prompt

```
<role>
You are an expert software engineer.
</role>
<goal>
Extract an abstract view from the classes keeping only the most important
classes.
</goal>

<description>
You will receive a component diagram in PlantUML format highlighting all
classes and their attributes and operations.
Additionally, the relationships between classes are also included in one form of
association which is not influential but helps you know which class connected to
which class.
</description>

<plantuml>
{plantuml_text}
</plantuml>
```

To evaluate the abstracted component diagram, a number of questions are proposed to systematically assess it:

- Q1) Did the LLM identified the correct components to represent the system at an appropriate level of abstraction?
- Q2) Are the relationships and interfaces between components correctly depicted, reflecting the system's intended interactions?
- Q3) Are the components and interactions correctly labeled?

Remarkably, the LLM demonstrated the ability to abstract away auxiliary utility classes and identify structurally and functionally central elements for both examples.

For the Coffee Machine, GPT-4o [26] selected: "*CoffeeMachine*", "*Boiler*", "*Display*", "*MachineTester*", and "*Controller*". Although "*MachineTester*" and "*Controller*" were not in the ground truth, upon inspection their inclusion was justifiable due to their functional relevance. Auxiliary classes

like "Boiler_boilWater" were correctly excluded. A similar abstraction quality was observed for the Dishwasher example, where the model returned "AbstractFactory", "Dishwasher", "Heater", "Jet", and "Tank". Only "AbstractFactory" was irrelative to the ground truth.

These results answer *Q1* and confirm the LLM's potential for filtering architectural noise and isolating conceptually meaningful components based solely on structural input.

Answering *Q2*: we can see in Fig. 10 and Fig. 12, the connection between the components are clear and understandable. However, the components "Controller" and "AbstractFactory" are isolated. This presentation is the result of the RE when we imported the source code into EA [20]. The RE did not find any connection from those two components to the other ones rendering those two to be isolated.

The LLM has not changed the names of the core components when it has selected them from the PlantUML text. Therefore, the component names came from the RE which is a reflection of the source code. Consequently, the relationship labels are also not impacted from the LLM, however, the RE did not call them with clear labels (i.e. "itsDisplay" or "itsHeater"). Answering *Q3*: The component names are understandable, but the connection labels are not.

The overall resulted component diagram is indeed an abstraction from the class diagram, which the LLM did an acceptable job with. The remaining factors are dependent on the RE process, which may be improved in the future.

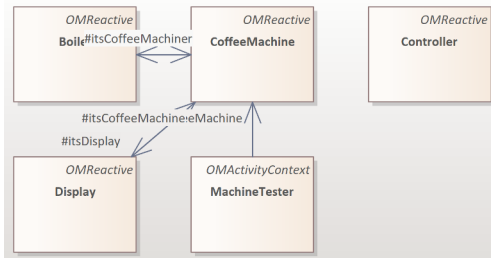


Fig. 5. Component diagram of the CoffeeMachine core components after abstraction using LLM

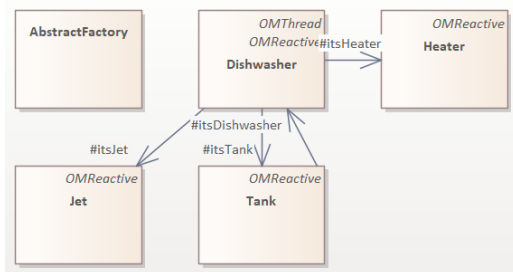


Fig. 6. Component diagram of the Dishwasher core components after abstraction using LLM

C. Behavioral View: State Machine Generation

1) *Prompting Strategy and Example Typology*: Following core component identification, we generated the behavioral view by passing the source code of each component to GPT-4o version 2024-11-20 [26] along with a structured prompt containing a solution plan. *Note: For better quality diagrams, refer to our repository.*

System Prompt

```
<Role>
You are an expert software engineer.
</Role>
<Goal>
Generate a state machine diagram for the c++ code in plantuml format.
</Goal>
<Solution Plan>
1. Understand the source code
2. Extract the candidate states
3. Provide small and summarized description of each state
4. Extract the transition from one state to another
5. What is the trigger that triggered the transition
6. Review the examples and how their corresponding state diagram
7. Construct the state diagram for the passed code
8. Review the order of the states based on normal logic and using the examples
</Solution Plan>
!! Important !!: The Controller example is customized to bring how parallel
states are present. Do not use it for any other reason.
```

Each component's logic was given in a single C++ script file, simplifying the input context. To enhance the model's understanding, we applied few-shot prompting [27], wherein pairs of source code (in form of text) and corresponding state machine diagrams were provided as illustrative examples. Since the ground truth state machine diagrams were provided as images, we pass them as such, however, we scaled them to a maximum width of 800px. This resolution keep all diagram details readable to the human eye and help in decreasing the token consumption.

Examples Prompt (repeat for each example)

```
Generate a state machine for the following code:
{source code} + {the ground truth state diagram as an image}
```

User Prompt

```
Generate a state machine for the following code:
{source code for the target class}
```

We categorized these few-shot examples into three types: 1) *General*: LLM-generated examples with simple behavioral logic: car door (showing the flow between Closed, Open, Locked states), freelance developer (how freelancer accept or decline an order), online retail (how a new order is paid, shipped, delivered, or cancelled flow). Because of the limited space, Fig. 7 show only the freelance general example, to show how the general examples are constructed. 2) *Expert*: Cross-example prompting using ground truth diagrams (e.g., Dishwasher states passed to guide Coffee Machine generation) and vice versa. 3) *Domain*: Prompting within the same domain,

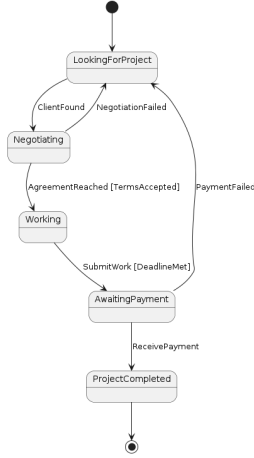


Fig. 7. General example (Freelance) generated using GPT-4o

where the state machine of one of the core components is generated using the other core components as examples.

These variations aimed to evaluate how different levels of expertise transfer influence to LLM's generation quality, where "Expert" examples are to simulate how an expert is passing his skill and influencing the output, "Domain" the experts' knowledge in addition to his skill, and "General" examples are just to assess if the LLM is capable of generating the behavior diagrams without the need for any influence from an expert.

2) *Evaluation Criteria and Scoring Scheme*: The focus of this paper is to prove a functional semi-automated approach for generating SAD. Accordingly, evaluating the generated state machine diagram is paramount to absolutely assess the functionality of the approach, and it does produce valid outputs. Thus, we propose a systematical scheme -based on our use case and can be generalized further- to assess the quality of the generated state machine diagrams based on the following key aspects:

- Q1) Can the LLM correctly identify the start and end states?
- Q2) Does the diagram include all ground truth states?
- Q3) Are the names of the generated states semantically accurate?
- Q4) Are all reference (ground truth) transitions present in the model?
- Q5) Are the triggers for each transition correctly inferred?
- Q6) Is every generated state reachable through at least one path (no dead states)?
- Q7) Are self-transitions correctly recognized and represented?
- Q8) Can the model capture parallel or concurrent state behaviors, when applicable?
- Q9) Does the overall diagram exhibit logical coherence and behavioral consistency in comparison to the ground truth?

Each question was scored as "X/Y (Z)", where "X" is the number of matched elements, "Y" is the total in the ground truth, and "Z" denotes the number of hallucinated elements, that are not found in the ground truth. A perfect match to the

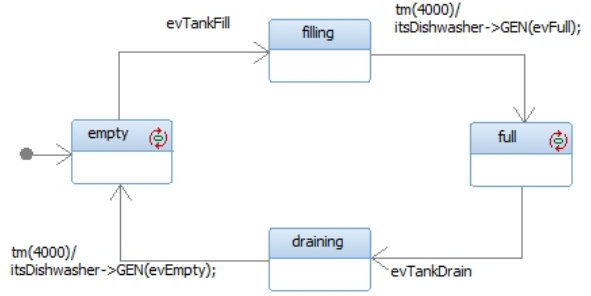


Fig. 8. Ground truth state diagram for the Tank component from Rhapsody

ground truth diagram is one where "X = Y" and "Z = 0". For instance: If a generated state diagram contains a total of 7 transitions and in the ground truth only 6 ones. Out of the 7 generated transitions, only 5 are matching, and the last two transitions are not present in comparison to the ground truth. Therefore, the score is shown as 5/6 (2).

3) *Comparative Analysis and Observations*: To ensure consistency and reliability, the state machine diagrams were generated multiple times. The evaluation was conducted manually by comparing the most representative output ³ against the ground truth. Based on the defined scoring criteria, the results for the "CoffeeMachine" and "Dishwasher" components are reported in Tables I and II, respectively.

We observed that smaller, less complex classes tend to yield higher-fidelity diagrams. In particular, classes such as "Display" and "Boiler" regarding the "CoffeeMachine", and "Heater", "Jet", and "Tank" regarding the "Dishwasher", exhibited strong alignment with the ground truth in terms of state coverage, transitions, triggers, and naming accuracy.

However, only "Heater", "Jet", and "Boiler" were acceptable diagrams when using general examples. Other classes, despite appearing close in numeric score, exhibited critical absence of the start states within substates in some cases and in another cases a hallucinated transition. These factors disrupted the flow and rendered the generated models behaviorally incomplete. For instance, as shown in Fig.10, the ground truth for the "CoffeeMachine" involves a transition from "off" to "on", followed by two parallel states. In the corresponding generated version (Fig.11), although the transition from "off" to "on" is captured, the entry point within the "on" composite state is missing, undermining the execution flow. Another one is "Tank" component where a hallucinated transition from "empty" to "draining" and the self transition in the state "full" corrupted the flow as shown in ground truth Fig.8 and the generated diagram Fig.9.

The two most complex component—"CoffeeMachine" and "Dishwasher"—presented the most challenges but also offered the richest insights. Their ground truth are shown in Figs. 10, 12, and generated diagrams are in, 11, and 13 respectively. Due to space limitations, not all diagrams are

³The file name with (picked) in the start of it.

TABLE I
COFFEE MACHINE STATE DIAGRAM SCORES

Component	Example	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9
Display	general	1/2 (0)	4/4 (0)	4/4 (0)	5/8 (0)	5/5 (0)	3/4 (0)	0/2 (0)	0/0 (0)	0/1 (0)
	expert	1/2 (1)	4/4 (0)	4/4 (0)	7/8 (1)	5/7 (0)	3/4 (0)	2/2 (0)	0/0 (0)	1/1 (0)
	domain	1/2 (1)	4/4 (0)	4/4 (0)	6/8 (1)	5/6 (0)	4/4 (0)	1/2 (0)	0/0 (0)	1/1 (0)
Boiler	general	2/2 (0)	6/6 (0)	4/6 (0)	4/11 (0)	4/4 (0)	6/6 (0)	0/1 (0)	0/0 (0)	0/1 (0)
	expert	2/2 (0)	6/6 (1)	4/6 (1)	10/11 (0)	8/10 (0)	6/6 (0)	0/1 (0)	0/0 (0)	1/1 (0)
	domain	2/2 (0)	6/6 (1)	4/6 (1)	10/11 (0)	8/10 (0)	6/6 (0)	0/1 (0)	0/0 (0)	1/1 (0)
CoffeeMachine	general	4/5 (0)	12/16 (0)	12/12 (0)	15/26 (0)	7/15 (0)	12/12 (0)	0/7 (0)	0/2 (0)	0/1 (0)
	expert	2/5 (0)	12/16 (1)	12/12 (1)	13/26 (2)	3/13 (0)	10/12 (0)	0/7 (0)	0/2 (0)	0/1 (0)
	domain	3/5 (0)	16/16 (3)	13/16 (6)	14/26 (3)	14/14 (0)	14/16 (0)	0/7 (0)	1/2 (0)	0/1 (0)

TABLE II
DISHWASHER STATE DIAGRAM SCORES

Component	Example	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9
Heater	general	1/1 (0)	2/2 (0)	2/2 (0)	3/3 (0)	3/3 (0)	2/2 (0)	0/0 (0)	0/0 (0)	1/1 (0)
	expert	1/1 (0)	2/2 (0)	2/2 (0)	3/3 (0)	3/3 (0)	2/2 (0)	0/0 (0)	0/0 (0)	1/1 (0)
	domain	1/1 (0)	2/2 (0)	2/2 (0)	3/3 (0)	3/3 (0)	2/2 (0)	0/0 (0)	0/0 (0)	1/1 (0)
Jet	general	1/2 (0)	3/4 (0)	3/3 (0)	5/5 (0)	5/5 (0)	3/3 (0)	0/0 (0)	0/0 (0)	1/1 (0)
	expert	1/2 (0)	4/4 (0)	4/4 (0)	7/7 (0)	7/7 (0)	4/4 (0)	0/0 (0)	0/0 (0)	1/1 (0)
	domain	1/2 (0)	3/4 (0)	3/3 (0)	5/5 (0)	5/5 (0)	3/3 (0)	0/0 (0)	0/0 (0)	1/1 (0)
Tank	general	1/1 (0)	4/4 (0)	4/4 (0)	5/5 (3)	5/5 (0)	4/4 (0)	0/0 (2)	0/0 (0)	0/1 (0)
	expert	1/1 (1)	4/4 (0)	4/4 (0)	5/5 (2)	5/5 (0)	4/4 (0)	0/0 (0)	0/0 (0)	0/1 (0)
	domain	1/1 (0)	4/4 (0)	4/4 (0)	5/5 (2)	5/5 (0)	4/4 (0)	0/0 (0)	0/0 (0)	0/1 (0)
Dishwasher	general	1/5 (0)	8/15 (0)	8/8 (0)	8/24 (0)	4/8 (0)	8/8 (0)	0/2 (0)	0/4 (0)	0/1 (0)
	expert	1/5 (0)	8/15 (0)	8/8 (0)	8/24 (2)	8/8 (0)	8/8 (0)	0/2 (0)	0/4 (0)	0/1 (0)
	domain	4/5 (0)	14/15 (0)	13/14 (1)	12/24 (0)	12/12 (0)	14/14 (0)	0/2 (0)	3/4 (0)	0/1 (0)

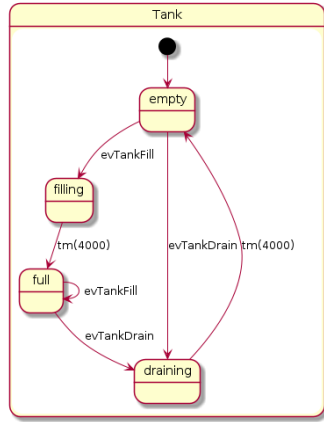


Fig. 9. Generated state diagram for the Tank component using GPT-4o

shown. We refer readers to our GitHub repository for full visual references [13].

In answering the evaluation questions, several patterns emerged:

- 1) Start states within substates were frequently missing, breaking the logical flow and violating the sequential execution expected in state hierarchies.
- 2) The generated diagrams sometimes adopted a hierarchical structure and other times a flat one. This variability means that what may appear as hallucinated states were, in fact, structural necessities to support hierarchy.

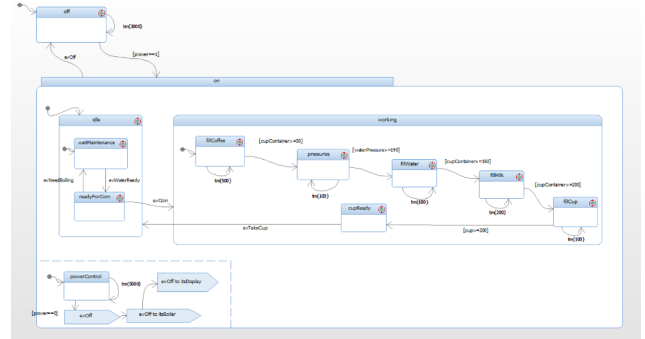


Fig. 10. Ground truth state diagram for the CoffeeMachine component from Rhapsody

3) State names generally reflected the terminology used in source code, indicating that the LLM leverages lexical cues from the implementation.

4) Transitions were largely captured, but self-transitions—often associated with timeouts—were almost entirely absent.

5) Timeout transitions were inconsistently labeled, frequently missing the actual time duration and instead annotated generically as *timeout*.

6) Parallel states were not reliably identified in general or expert examples. However, domain-specific examples substantially improved recognition of concurrent behaviors.

7) Certain state types, such as the "*H**" pseudostate and

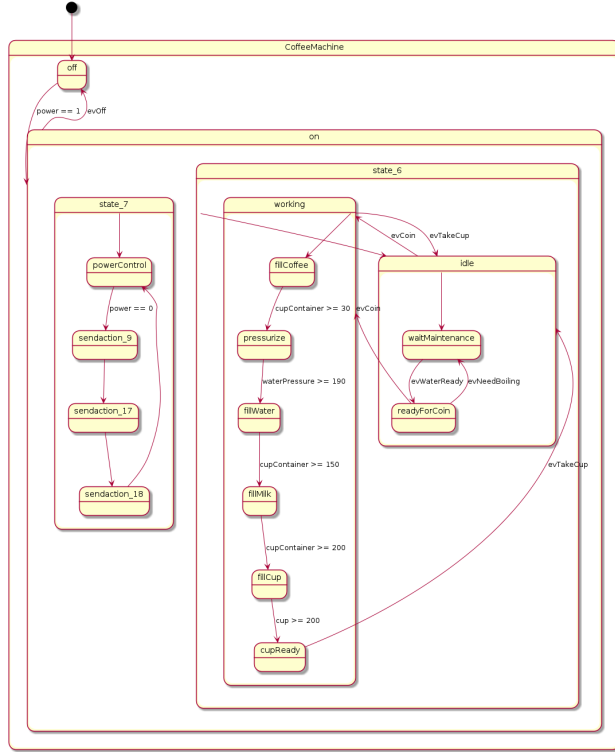


Fig. 11. Generated state diagram for the CoffeeMachine component using GPT-4o

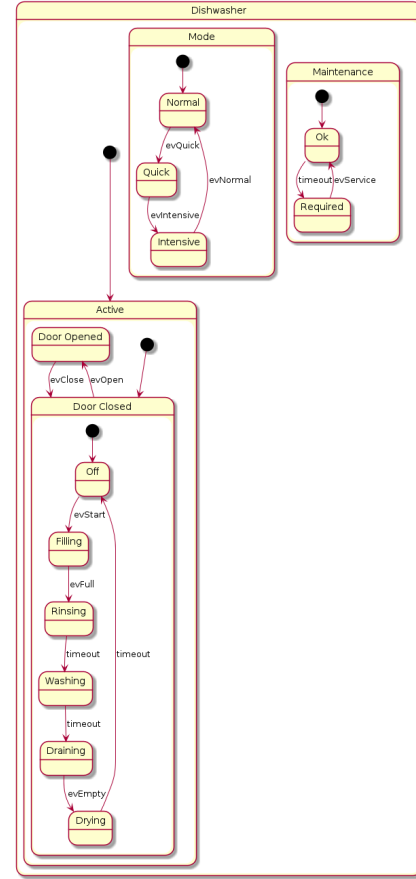


Fig. 13. Generated state diagram for the Dishwasher component using GPT-4o

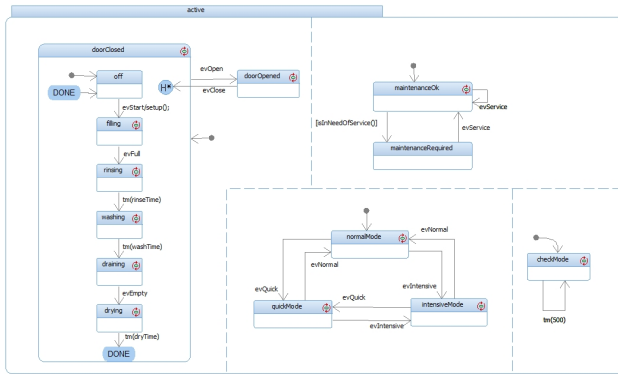


Fig. 12. Ground truth state diagram for the Dishwasher component from Rhapsody

final state "Done" in the *Dishwasher* class, were entirely omitted.

8) As a consequence of these issues—particularly the missing substate entry points—the overall behavioral logic for the largest classes was deemed unsatisfactory, despite domain-specific prompting notably improved the LLM generation quality.

Different example types used during generation yielded varied levels of quality. Diagrams generated using domain-specific examples showed the strongest alignment with ground

truth, successfully modeling complex behaviors such as parallelism. In contrast, expert examples were adequate only for less complex cases, while general examples performed poorly in all dimensions other than the simple straight forward ones.

These findings suggest that while GPT-4o [26] possesses a foundational ability to infer behavioral models from code, this capability is highly sensitive to the structure and semantic relevance of the input examples. Specifically, incorporating domain-aligned few-shot prompts is essential for achieving accurate, usable behavioral diagrams in real-world projects.

V. CONCLUSION AND FUTURE WORK

This work proposed a hybrid approach for semi-automatically generating SAD from source code by combining RE with LLM-guided abstraction. The method effectively recovered both static and behavioral views, with the LLM identifying core components and generating meaningful state machine diagrams with the help of the proposed systematic assessment.

Reliance on LLM-guided abstraction for core component detection showed a high promising usability. However, the generalizability remains an open point.

Moreover, the generation of the state machine diagram using domain-specific examples notably improved behavioral modeling, implicitly injecting expert knowledge into the process. The pre-trained LLM also trained on base knowledge that is improving over time and might overcome the need for high quality domain examples.

Nonetheless, a consistent challenge is the context window, some components could have a huge code that exceed the context windows, requiring a technique keeping the contextual relevant parts for the LLM to correctly infer the component's behavior such as iteratively prompt the source code and build the state diagram step by step.

Addressing the research questions, our approach showed the ability to semi-automate the process of generating static and behavioral diagrams, where the RE step is manually done, and the LLM is automated. Moreover, the LLM with the help of the domain examples did infer the code behavior and translate it to a state machine diagram. However, the high complexity digrams suffered from being logically flawed. And the lesser complex ones had a better, more human like diagrams.

Future work should also focus on improve behavioral inference, promising directions include using LLM agents [28], and integrating reasoning-capable models like DeepSeek [29] or Phi-4 [30] to better capture implicit logic, state hierarchies, and parallel states. These enhancements aim to increase the precision, flexibility, and autonomy of automated SAD generation.

ACKNOWLEDGMENT

This research was made possible through the generous support of IAV GmbH, whose contributions are deeply appreciated. Heartful thanks to Damian Hofmann, Christoph Raab, and Christian Nabert for their constructive engagement and beneficial discussions, which significantly contributed to the reported findings. Another acknowledgment goes to generative AI namely ChatGPT and Google Gemini for assisting in reviewing its language, grammar, and restructure some paragraphs.

REFERENCES

- [1] M. Gharbi, A. Koschel, and A. Rausch, *Software Architecture Fundamentals*. Rocky Nook, Inc., 2020.
- [2] D. Rost, M. Naab, C. Lima, and C. von Flach Garcia Chavez, "Software architecture documentation for developers: A survey," in *Software Architecture*, K. Drira, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 72–88.
- [3] C. Satish and M. Anand, "Software documentation management issues and practices: A survey," *Indian Journal of Science and Technology*, vol. 9, no. 20, pp. 1–7, 2016.
- [4] "What Are the Consequences of Poor Documentation in Projects? A Deep Dive + Prevention Tips — itsdart.com," <https://www.itsdart.com/blog/the-consequences-of-poor-documentation-in-projects#:~:text=Poor%20documentation%20in%20projects%20can,compromised%20quality%2C%20and%20reputational%20damage.,> [Accessed 23-05-2025].
- [5] S. A. Rukmono, F. Zamfirov, L. Ochoa, and M. Chaudron, "From expert to novice: An empirical study on software architecture explanations," *arXiv preprint arXiv:2503.08628*, 2025.
- [6] E. Chikofsky and J. Cross, "Reverse engineering and design recovery: a taxonomy," *IEEE Software*, vol. 7, no. 1, pp. 13–17, 1990.
- [7] A.-M.-C. Nicolaescu, "Verhaltensbasierte Architekturkonformitätsüberprüfung," *Ausgezeichnete Informatikdissertationen*, vol. 2018, pp. 261–270, 2019. [Online]. Available: <https://publications.rwth-aachen.de/record/808671>
- [8] D. Nam, A. Macvean, V. Hellendoorn, B. Vasilescu, and B. Myers, "Using an llm to help with code understanding," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: <https://doi.org/10.1145/3597503.3639187>
- [9] V. Punugupati, "Leveraging LLMs for software architecture diagrams: A practical guide," <https://www.linkedin.com/pulse/leveraging-llms-software-architecture-diagrams-guide-punugupati-2r6ne/>, Nov. 2024, accessed: 2025-5-20.
- [10] M. Sharma, "How I use chatgpt to create UML diagrams saving me ton of time," <https://medium.com/@mayank.sharma2796/how-i-use-chatgpt-to-create-uml-diagrams-saving-me-ton-of-time-e94fde009b12>, Dec. 2024, accessed: 2025-5-20.
- [11] W. Ma, S. Liu, Z. Lin, W. Wang, Q. Hu, Y. Liu, C. Zhang, L. Nie, L. Li, and Y. Liu, "Lms: Understanding code syntax and semantics for code analysis," *arXiv preprint arXiv:2305.12138*, 2023.
- [12] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, pp. 1–79, 2024.
- [13] "GitHub: Generating software architecture description from source code using reverse engineering and large language model," <https://github.com/ahmadhatahet/generate-software-architecture-description-using-llm>, [Accessed 07-08-2025].
- [14] M. Schiewe, J. Curtis, V. Bushong, and T. Cerny, "Advancing static code analysis with language-agnostic component identification," *IEEE Access*, vol. 10, pp. 30 743–30 761, 2022.
- [15] G. Granchelli, M. Cardarelli, P. Di Francesco, I. Malavolta, L. Iovino, and A. Di Salle, "Towards recovering the software architecture of microservice-based systems," in *2017 IEEE International conference on software architecture workshops (ICSAW)*. IEEE, 2017, pp. 46–53.
- [16] "Doxygen," <https://www.doxygen.nl/>, [Accessed 20-05-2025].
- [17] "SmartDraw Create Flowcharts, Floor Plans, and Other Diagrams on Any Device," <https://www.smartdraw.com/>, accessed: 2025-5-20.
- [18] B. Kryza, "clang-uml: Customizable automatic UML diagram generator for c++ based on clang."
- [19] "Open-source tool that uses simple textual descriptions to draw beautiful UML diagrams," <https://plantuml.com/>, accessed: 2025-5-20.
- [20] "Sparxsystems.com. (2019). uml modeling tools for business, software, systems and architecture." <https://sparxsystems.com/>, [Accessed 20-05-2025].
- [21] A. Walker, I. Laird, and T. Cerny, "On automatic software architecture reconstruction of microservice applications," in *Information Science and Applications: Proceedings of ICISA 2020*. Springer, 2021, pp. 223–234.
- [22] D. Amalfitano, M. De Luca, D. F. De Angelis, and A. R. Fasolino, "Automated architecture recovery for embedded software systems: An industrial case study," in *Software Architecture*, M. Galster, P. Scandurra, T. Mikkonen, P. Oliveira Antonino, E. Y. Nakagawa, and E. Navarro, Eds. Cham: Springer Nature Switzerland, 2024, pp. 53–68.
- [23] Y. Zhang, Z. Xu, C. Liu, H. Chen, J. Sun, D. Qiu, and Y. Liu, "Software architecture recovery with information fusion," ser. ESEC/FSE 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 1535–1547. [Online]. Available: <https://doi.org/10.1145/3611643.3616285>
- [24] S. P. R. Puchala, J. K. Chhabra, and A. Rathee, "Ensemble clustering based approach for software architecture recovery," *International Journal of Information Technology*, vol. 14, no. 4, pp. 2013–2019, 2022.
- [25] A. Sajji, Y. Rhazali, and Y. Hadi, "A methodology of automatic class diagrams generation from source code using model-driven architecture and machine learning to achieve energy efficiency," in *E3S Web of Conferences*, vol. 412. EDP Sciences, 2023, p. 01002.
- [26] A. Hurst, A. Lerer, A. P. Goucher, A. Perelman, A. Ramesh, A. Clark, A. Ostrow, A. Welihinda, A. Hayes, A. Radford *et al.*, "Gpt-4o system card," *arXiv preprint arXiv:2410.21276*, 2024.
- [27] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [28] "LLM agents," <https://www.promptingguide.ai/research/llm-agents>, accessed: 2025-5-20.

- [29] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi *et al.*, “Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning,” *arXiv preprint arXiv:2501.12948*, 2025.
- [30] M. Abdin, J. Aneja, H. Behl, S. Bubeck, R. Eldan, S. Gunasekar, M. Harrison, R. J. Hewett, M. Javaheripi, P. Kauffmann *et al.*, “Phi-4 technical report,” *arXiv preprint arXiv:2412.08905*, 2024.